

Spatial Atlas: Compute-Grounded Reasoning for Spatial-Aware Research Agent Benchmarks

From computable evidence to adaptive spatial actions

Arun Sharma | Advisors: Prof. Shashi Shekhar and Prof. Vipin Kumar
Department of Computer Science and Engineering, University of Minnesota, Twin Cities

1. Overview

Objective: Compute deterministic subproblems before language generation (**CGR**).

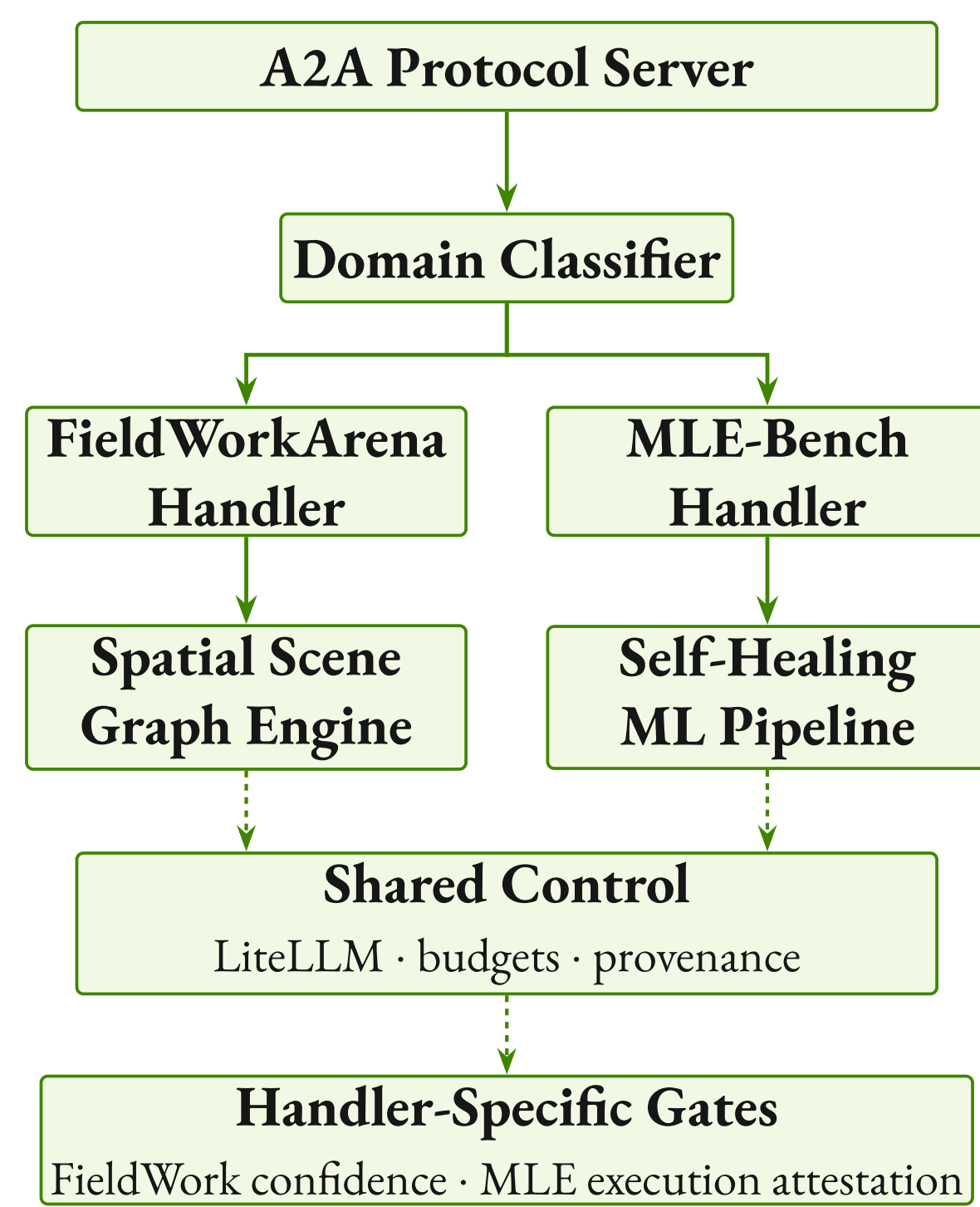
Limit: Answer-only scores cannot distinguish grounded geometry from guessing.

KEY TAKEAWAY

Computing what can be computed before generating what must be generated.

2. Spatial Atlas System

Two benchmark-oriented handlers share one A2A server, model routing, budgets, and provenance controls.



Approach and why it will succeed: Four mechanisms instantiate compute-grounded reasoning (CGR) end to end.

- **Typed scene evidence:** entities, estimated positions, relations, zones, and constraints.
- **Confidence-gated reflection:** one conditional refinement when self-graded confidence is below 0.6.
- **Fail-closed execution:** generated code requires explicit opt-in and isolated-worker attestation.
- **Repair then score-gated refinement:** diagnose and patch failures using error and stdout traces. After successful execution, retain a refinement only when its parsed validation score improves.

The spatial branch computes relations and facts. The ML branch analyzes tasks, generates and repairs pipelines, and compares pipeline-emitted validation scores.

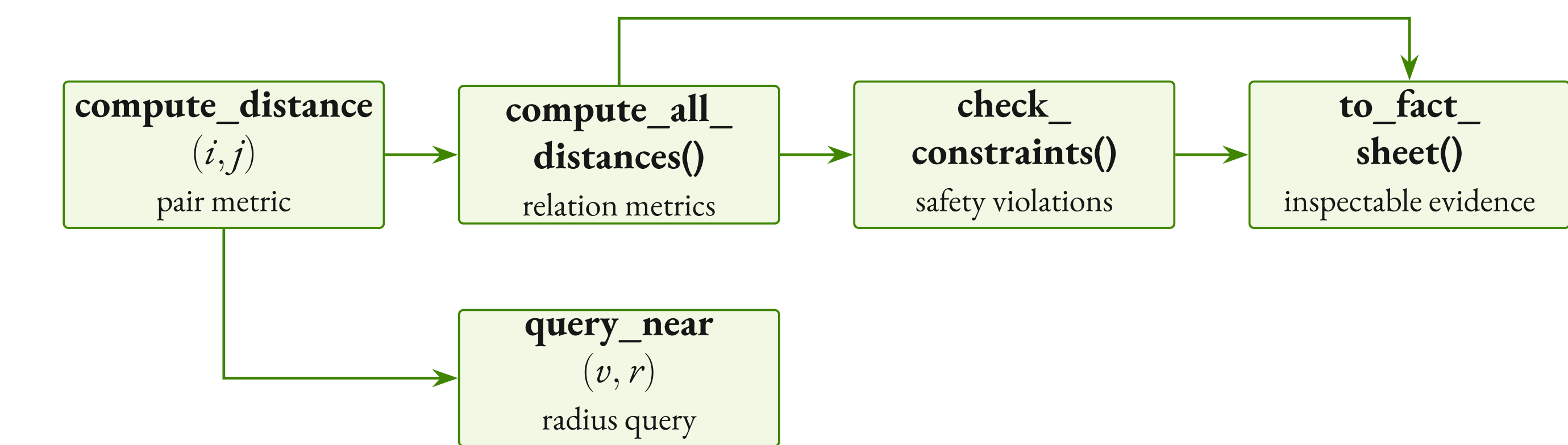
Figure 1. Spatial Atlas architecture. A classifier routes tasks to two handlers that use shared control services and apply handler-specific gates. Dashed lines indicate shared services.

3. Contributions

Computable evidence substrate: Given an image I and question q , CGR separates evidence construction from answer generation by creating a typed graph $G = (V, E)$:

$v_i = \text{SpatialEntity}(\text{id}_i, \ell_i, \text{pos}_i, \text{attrs}_i, z_i)$ (1) $e_{ij} = \text{SpatialRelation}(i, p_{ij}, j, d_{ij})$ (2)

Deterministic graph program: The graph exposes four computation and query methods plus one fact-sheet serializer over vision-language-estimated coordinates, producing structured evidence rather than measured 3D geometry.



Integration rationale: Four SpatialClaw mechanisms enable stateful CGR.

- **Code as action:** AST-checked cells compose tools one step at a time.
- **Persistent state:** variables and outputs persist across steps.
- **Metric perception:** SAM3 and Depth-Anything-3 expose masks and reconstructed geometry.
- **Feedback-driven revision:** stdout, variables, and images inform the next step before ReturnAnswer.

6. SpatialClaw Integration

Integration design: Spatial Atlas specifies the evidence that should be available before generation. SpatialClaw contributes a stateful code-action interface that can inspect tool outputs, compose operations, and revise computation after each observation.

Require: task T , state $\mathcal{K}_0$, budget B , threshold τ

```

1  $a_0, \sigma_0 \leftarrow \text{FastModel}(T, \mathcal{K}_0)$ 
2 if  $\sigma_0 \geq 0.8$  then
3   return  $a_0$ 
4 end if
5  $\mathcal{K}_1 \leftarrow \mathcal{K}_0 \cup \text{SceneGraph}(T)$ 
6  $a_1, \sigma_1 \leftarrow \text{StandardModel}(T, \mathcal{K}_1)$ 
7 for  $r = 1$  to 2 do
8   if  $\sigma_1 \geq \tau$  then
9     return  $a_1$ 
10  end if
11   $\mathcal{K}_{r+1} \leftarrow \text{Reflect}(\mathcal{K}_r, a_1)$ 
12   $a_1, \sigma_1 \leftarrow \text{StrongModel}(T, \mathcal{K}_{r+1})$ 
13 end for
14 return  $a_1$ 

```

4. Evaluation

FieldWorkArena Handler: The v2 preprint describes a pipeline using Florence-2, GPT-4.1 vision, and bounding-box-centroid positions. The current repository instead obtains `position_x` and `position_y` from a Strong-model JSON extraction prompt.

- 1 Describe the image and extract entities and relations.
- 2 Store a typed scene graph $G = (V, E)$.
- 3 Compute pairwise distances and safety-rule violations.
- 4 Pass a fact sheet to the final language model.

MLE-Bench Handler: Execution of generated code follows a fail-closed policy that requires explicit operator opt-in and isolated-worker attestation.

- 1 Analyze task type, metric, files, and constraints.
- 2 Generate a strategy-specific Python pipeline.
- 3 Run only inside the bounded worker after the gate.
- 4 Diagnose, patch, and retry failures for at most 3 attempts.
- 5 Parse VALIDATION_SCORE after success.
- 6 Refine at most twice and retain only a score improvement.

The rollback rule is explicit:

$$p_{t+1} = \begin{cases} p' & \text{better}(s(p'), s(p_t)) \\ p_t & \text{otherwise} \end{cases} \quad (6)$$

Prompted safeguards: The generated pipeline is instructed to check ID overlap, row fingerprints, temporal order, and byte-identical media. These checks are not independently enforced. Dummy fallback remains opt-in.

5. Source-Reported Results

Table 1: FieldWorkArena Accuracy (%)				Table 2: MLE-Bench Performance (%)			
Configuration	Factory	Warehouse	Retail	Category	Valid	Medal	Competitions
w/o Spatial Scene Graph	51	44	55	Tabular	91	42	32
w/o Entropy-Guided Reasoning	65	60	67	NLP	78	28	18
w/o Florence-2	63	58	66	Vision	65	15	12
GPT-4V Direct	48	41	52	Time Series	85	35	8
				Other	72	20	5
Overall	72	68	74	Overall	82	32	75

REPORTED RESULT

Removing the Spatial Scene Graph changes Factory, Warehouse, and Retail accuracy by -21 , -24 , and -19 points. MLE-Bench reports **82% valid submissions** and **32% medals** across 75 competitions.

Limitation without SpatialClaw: Atlas computes deterministic relations over model-estimated 2D coordinates. Localization and depth errors can therefore propagate into distances and constraints.

Where SpatialClaw works best: Multi-step tasks with reliable masks, depth, or point maps that require geometric inspection, operation composition, evidence recomputation, and answer revision.

SCENE-GRAPH DATAFLOW DAG

- **Scene-graph flow:** Extraction proposes entities and approximate positions $\rightarrow$ structuring writes typed vertices and relations $\rightarrow$ computation evaluates geometry and constraints on the fixed graph. `compute_distance(i, j)` feeds the bulk path `compute_all_distances()` $\rightarrow$ `check_constraints()` $\rightarrow$ `to_fact_sheet()`, which serializes stored distances and violations. It also feeds the independent read-only `query_near(v, r)` radius branch, which is not required for serialization.

- **Entropy-Guided Reasoning:** At step t , the knowledge state $\mathcal{K}_t$ aggregates observations, computed facts, and intermediate conclusions. The paper defines answer entropy and selects the action with the greatest expected entropy reduction:

$$H(\mathcal{A} \mid \mathcal{K}_t) = - \sum_{a \in \mathcal{A}} P(a \mid \mathcal{K}_t) \log P(a \mid \mathcal{K}_t) \quad (3)$$

$$c^* = \arg \max_{\hat{c}} \mathbb{E}[H(\mathcal{A} \mid \mathcal{K}_t) - H(\mathcal{A} \mid \mathcal{K}_t \cup \text{obs}(c_j))] \quad (4) \quad \text{reflect}(a) = \begin{cases} \text{True}, & \sigma(a) < \tau \\ \text{False}, & \text{otherwise} \end{cases} \quad (5)$$